Blackbody Heat Model

Christian Gonzalez *

September 26, 2025

This model aims to represent the temperature on a blackbody's surface. In MATLAB, ode45 is used to find the solution to the heat equation in the x, y, and t dimensions. Importantly, there are three main sources of heat that are modeled in the script: blackbody radiation, diffusion across the blackbody's surface, and heat flow from the [heaters].

0.1 Radiation and Controller Terms

This section will describe **radiation(T)**, **ctrl_term(targetTemp, T)**, and its helper functions. The following equation is used to calculate the derivative of temperature with respect to time:

$$\frac{\mathrm{d}T_{\mathrm{BB}}}{\mathrm{d}t} = \frac{-A_p\sigma}{m_p c_p}\left(T_{\mathrm{BB}}^4 - T_\infty^4\right) + c\left(T_c - T\right) \tag{1}$$

This equation describes $\frac{dT_{\mathrm{BB}}}{dt}$ for a point mass according to the Stefan-Boltzmann law and a proportional controller term where c is the gain coefficient and $targetTemp$ is the set point. For the sake of the ode45 solver, constants $M_p c_p$ are consolidated into a single variable named **sb**, which is defined by the function **stefan_const(T)**. Modifications to physical constants such as area, emissivity, mass, etc. can be made in this helper function.

ctrl_term(targetTemp, T) has a **gain coefficient (c)** that changes the controller's behavior. A higher c-value will make it converge to the set point at a faster rate,

*cg5739@princeton.edu

whereas a low c value will make it converge at a slower rate (or not at all). The contribution Q controller term is capped at both a max and a min, which are defined as $Q_{\max}$ and $Q_{\min}$, respectively. radiation(T) is programmed as it appears in eq. 1, and may only require modification if T_∞ is not 293 K.

0.2 Logic for Spatial [Heater] Modeling

This model includes logic that constructs a grid of [heaters] that provide heat for the blackbody. This work is done by the function **index_calc(xlen, ylen)**. The [heater's] dimensions can be accessed through variables $blockSize$ and GAP, which define the physical widths in units of index. Used in conjunction with a for loop, it is possible to keep track of which *indices* come into contact with [heaters] (as opposed to (x,y) locations).

0.3 Diffusion

The most complicated part of this script is the calculation that is done for diffusion. Heat change according to Fourier's law follows this rule:

$$\kappa \nabla^2 T \tag{2}$$

ode45 is able to calculate the rate of change in temperature according to this law in two dimensions of space, but it requires a very specific setup (this is perhaps the most complicated part of the script). Using the definition of the second partial derivative, we are able to turn this problem into an ODE. In the model, the heat equation for a specific cell with position (i,j) is as follows:

$$\kappa \left[\frac{T_{i-1,j} - 2T_{i,j} + T_{i+1,j}}{(\Delta x)^2} + \frac{T_{i,j-1} - 2T_{i,j} + T_{i,j-1}}{(\Delta y)^2} \right] \tag{3}$$

The numerators here represent a total dT across the cells immediately adjacent to the cell (i,j). The two terms in this expression correspond to two sets of directions: up and down, left and right. It turns out that the equation is made much simpler by equating $\Delta x = \Delta y = h$, giving us the form that is solvable through ode45:

$$\rho \delta z \frac{\mathrm{d}T_{i,j}}{\mathrm{d}t} = \frac{\kappa}{h^2} \left[T_{i-1,j}(t) + T_{i,j-1}(t) - 4T_{i,j}(t) + T_{i+1,j}(t) + T_{i,j-1}(t) \right] \tag{4}$$

Programmatically, the problem becomes simple. The goal is to find an operation that turns a temperature vector into the bracketed term on the right side of Equation (4). This is done through a carefully constructed matrix that can distribute out coefficients as seen in the bracketed expression (e.g. -4 in $-4T_{i,j}(t)$). Eq. 4 represents a case where there are 4 neighboring cells. However, not every cell has 4 neighboring cells, therefore edge detection logic must be implemented.

0.4 Developing the Matrix

The construction of this matrix operator (using **nabla(xlength, ylength)**) is based in edge detection logic. Before explaining the process of its construction, it is important to mention that T is a column vector with dimensions $rows = Nx * Ny, columns = 1$. Because we want to map to a column vector $\frac{dT_{i,j}(t)}{dt}$ with the same dimensions, our matrix must be a square matrix with $rows = columns = Nx * Ny$. The process is as follows (note that this is **diffusion(T, xlength, ylength, h, nabla**$_operator$) $:to$

$c\nabla\vec{T(t)} = \frac{d\vec{T(t)}}{dt}$ $\quad (c[NxNy, NxNy] \cdot [NxNy, 1] = [NxNy, 1])$(5) The nabla matrix is calculated using the above concept/pattern - a cell with 4 neighboring cells has 4 contributions to its diffusion. Thus, neighboring cells must be assigned a coefficient of $+1$ and the cell (i,j) must be assigned a coefficient -4 to calculate the difference. In general, this is true. If there are three neighboring cells, then cell (i,j) must be assigned a coefficient -3. So the job is to: assign a coefficient of $+1$ to the indices for neighboring cells, keep track of how many neighboring cells there are, and finally assign the cell (i,j) with coefficient $-count$ where count is the number of neighboring cells.

There is a loop in the code (variable n) which keeps track of (i,j). There are four if statements, corresponding to each direction (up, down, left and right), which do the job described above (check neighboring cells and assign values accordingly). Results are successful as seen below:

```
% TXY, where X is row and Y is column
syms T11 T12 T13 T21 T22 T23 T31 T32 T33
T           = [T11;T12;T13;T21;T22;T23;T31;T32;T33];
dTdt        = nabla(3, 3) * (T);
```

```
% print results
for i = 1:length(T)
display(char(T(i)) + "  -->  " + char(dTdt(i)));
end

% Output:
    "T11  -->  T12 - 2*T11 + T21"
    "T12  -->  T11 - 3*T12 + T13 + T22"
    "T13  -->  T12 - 2*T13 + T23"
    "T21  -->  T11 - 3*T21 + T22 + T31"
    "T22  -->  T12 + T21 - 4*T22 + T23 + T32"
    "T23  -->  T13 + T22 - 3*T23 + T33"
    "T31  -->  T21 - 2*T31 + T32"
    "T32  -->  T22 + T31 - 3*T32 + T33"
    "T33  -->  T23 + T32 - 2*T33"
```

This shows $\vec{T(t)}$, and its corresponding $\frac{d\vec{T(t)}}{dt}$. This works for any dimensions of discretization.

0.5 Breaking Points and Next Steps

One important breaking point for this model is the ratio of $\Delta x = \Delta y = h$ to Δt - it must obey the **CFL condition**. As it stands, the model has a ratio between change h and change t, which seems to be appropriate for the discretizations needed (though it hasn't been evaluated with respect to the CFL condition). Deviation from this current ratio will cause it to break. One thing worth noting here is that the x and y axes are defined by **del_X , del_Y** and **xN, yN**. These do not define the number of x and y discretizations, but rather define the step size (del_X) and the upper range (xN).